

Furkan Büyüksarıkulak

ID 22002097

EE-102, Section 003

Lab 4 : Arithmetic Logic Unit (ALU)

Experiment 4

25.10.2023

AIM:

This lab project aimed to create an arithmetic logic unit (ALU) that performs eight functions ; addition, subtraction, AND_gate, OR_gate, XOR_gate, right rotate, right logical shifter, complement . This was implemented our VHDL code on the Basys3 and observe the result and behaviour of the ALU calculator via LED's.

DESIGN SPECIFICATIONS:

Initially, we first had to decide which eight operations we would use and how many bits to functionalize. ALU is designed as a hierarchial namely it has several sub modules. This design structure provides us with practicality and ease of use among operators. The operators I chosen for the ALU are as in (Table1).

<i>Addition</i>
<i>Substraction</i>
<i>AND Gate</i>
<i>OR Gate</i>
<i>XOR Gate</i>
<i>Right Rotate</i>
<i>Right Logical Shift</i>
<i>Complement</i>

Table1 : Opeartors

In addition to the submodules of these opeators, we have also have half adder and full adder submodules. These submodules were used to design submodules which are addition and subtraction. The half adder and full adder diagrams are shown in (Figure1) and (Figure2), respectively.

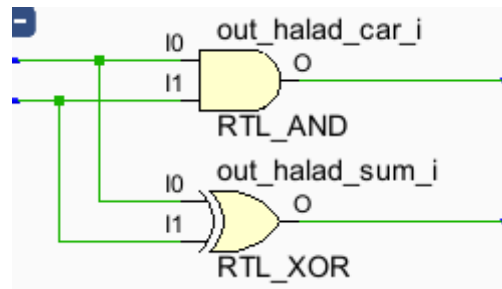


Figure1 : Half Adder Schematic

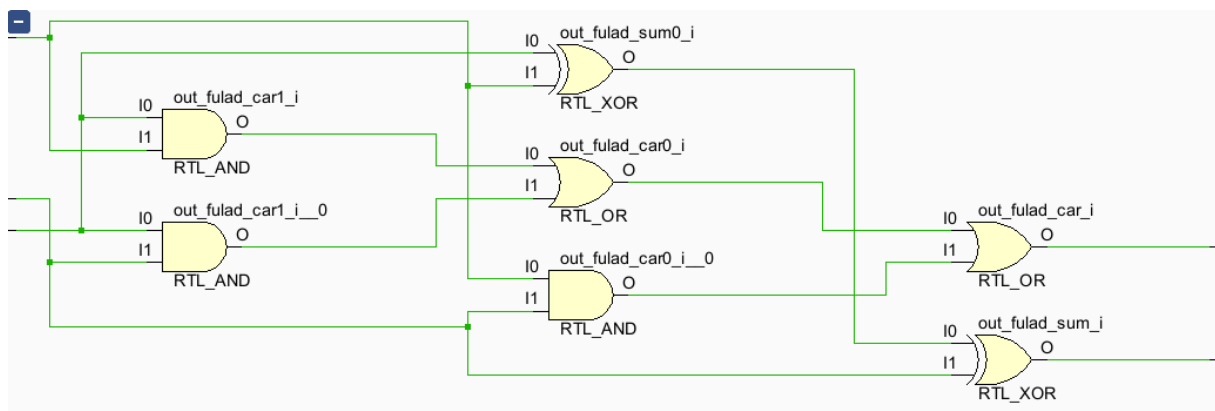


Figure2 : Full Adder Schematic

Half adder takes in_halad_a and in_halad_b as inputs and gives out_halad_sum and out_halad_car as outputs. It performs 1-bit addition by using “AND” and “XOR” gates, then the half adder gives a result and a carry.

Full adder takes inputs as in_fulad_a, in_fulad_b, then as a carry in_fulad_c which can come from another operation. As outputs it gives out_fulad_sum and out_fulad_car. It performs 1-bit addition by using “AND”, “XOR” and “OR” gates. Moreover, it gives a result and a carry.

The submodule that performs the addition of binary numbers is coded as addition operation. It uses 1 half adder and 3 full adder to perform this process. Addition operation takes 4 bit a and b input as a0_in, a1_in, a2_in, a3_in, b0_in, b1_in, b2_in, b3_in and it is resulted with 5 bit output as add_out. The design of this sub module can be seen in (Figure3).

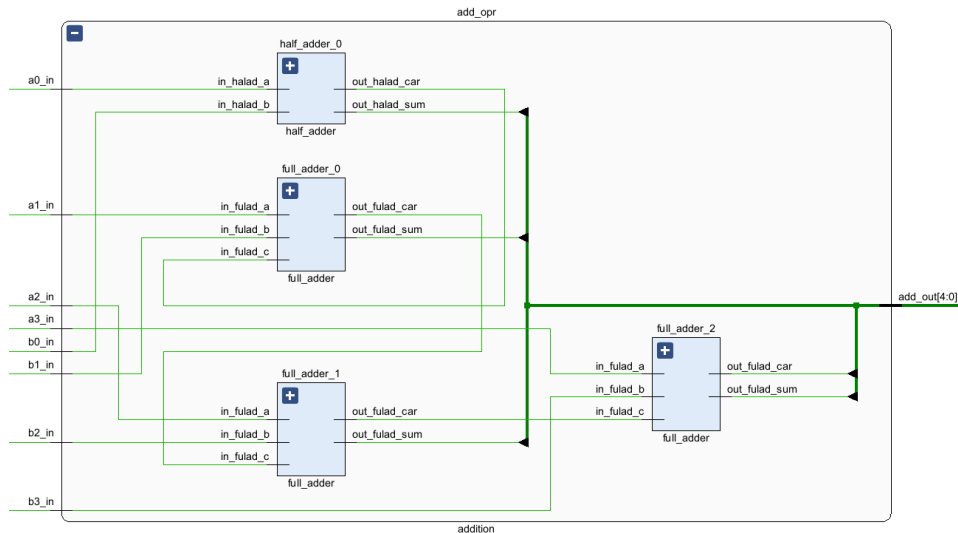


Figure3 : Addition Operation Schematic

The submodule that performs subtraction of binary numbers is coded as subtraction operation. The logic used when doing this operation uses input a and inverse of input b, that is ; $a + (\text{not } b) + 1 = a - b$. It uses 4 full adder to perform this process. Substraction operation takes 4 bit a and b input as a0_in, a1_in, a2_in, a3_in, b0_in, b1_in, b2_in, b3_in and it is resulted with 5 bit output as sub_out, and it does 4 bit signed subtraction on binary numbers. The design of this sub module can be seen in (Figure4).

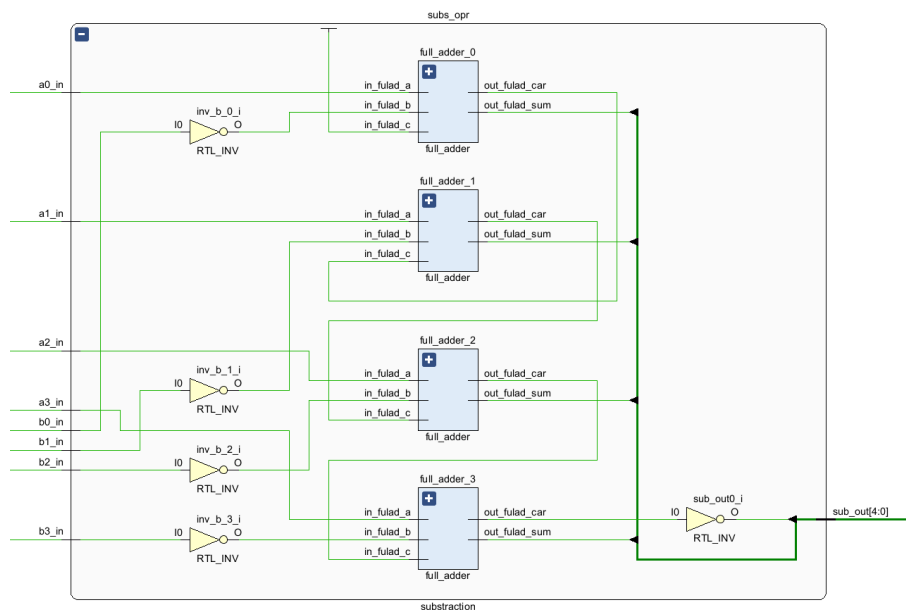


Figure4 : Substraction Operation Schematic

The submodule representing the bitwise AND logic gate is coded as AND gate operation. This gate operation takes 4 bit a and b input as a0_in, a1_in, a2_in, a3_in, b0_in, b1_in, b2_in, b3_in and it is resulted with 4 bit output as and_out. It uses 4 AND gates to perform this operation for each bit and it give the result for each bit using AND gates. The design of this sub module can be seen in (Figure5).

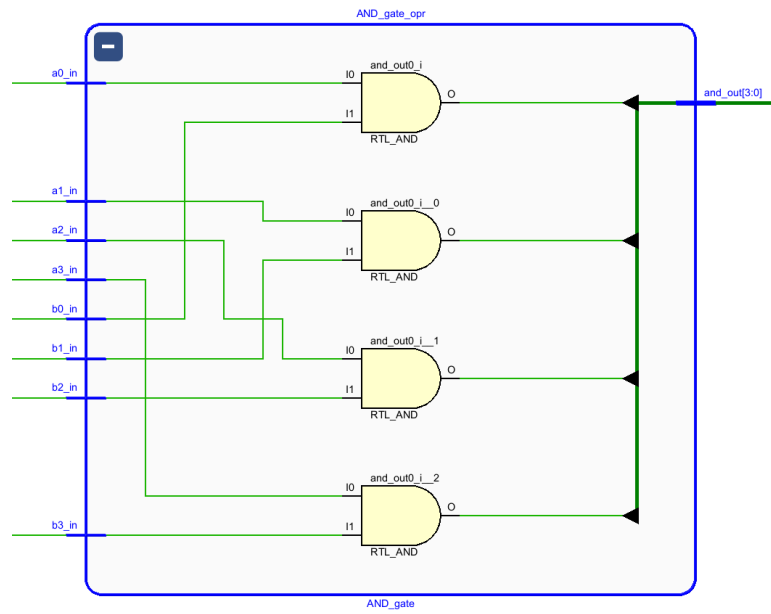


Figure5 : AND Gate Operation Schematic

The submodule representing the bitwise OR logic gate is coded as OR gate operation. This operation takes 4 bit a and b input as a0_in, a1_in, a2_in, a3_in, b0_in, b1_in, b2_in, b3_in and it is resulted with 4 bit output as or_out. It uses 4 OR gates to perform this operation for each bit and it give the result for each bit using OR gates. The design of this sub module can be seen in (Figure6).

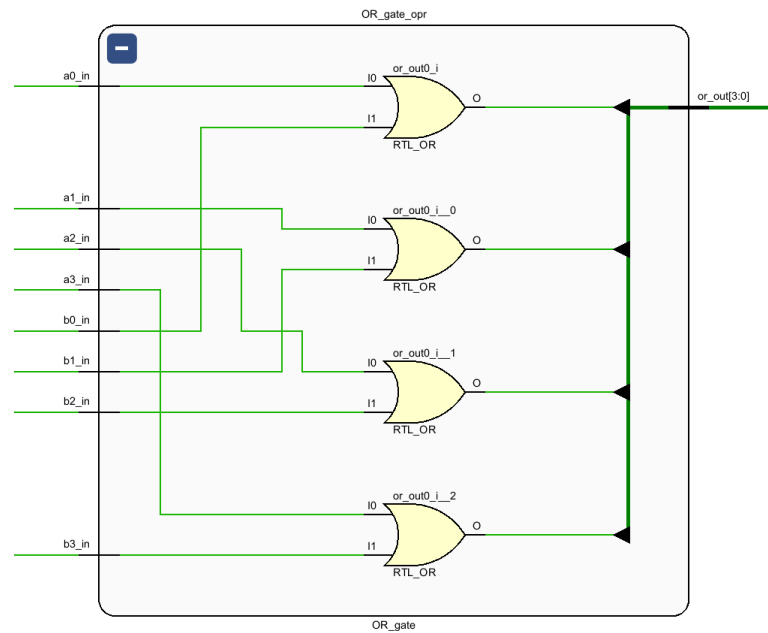


Figure6 : OR Gate Operation Schematic

The submodule representing the bitwise XOR logic gate is coded as XOR gate operation. This operation takes 4 bit a and b input as a0_in, a1_in, a2_in, a3_in, b0_in, b1_in, b2_in, b3_in and it is resulted with 4 bit output as xor_out. It uses 4 XOR gates to perform this operation for each bit and it give the result for each bit using XOR gates. The design of this sub module can be seen in (Figure7).

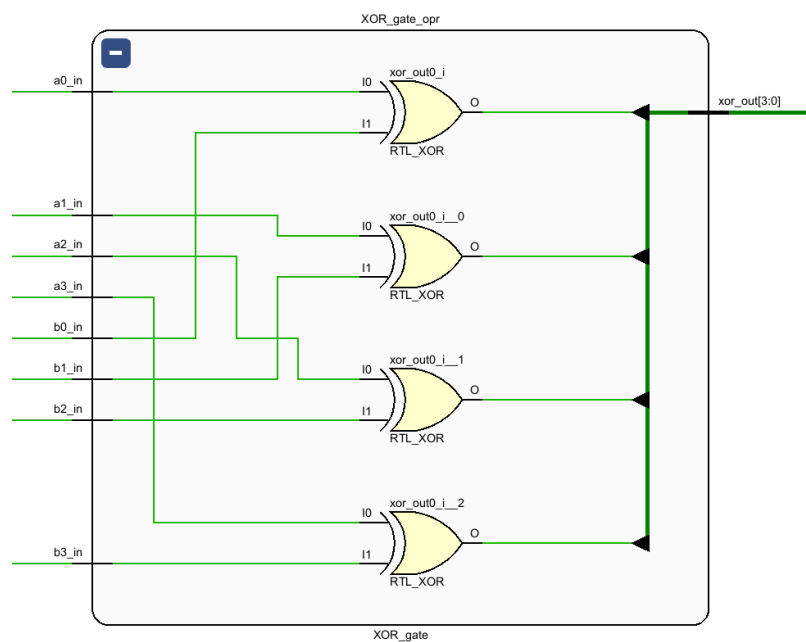


Figure7 : XOR Gate Operation Schematic

The right rotate operation submodule performs shifting each bit to the right and inserts the right most bit to the left most bit. Right rotate operation takes 4 bit 'a' input as a0_in, a1_in, a2_in, a3_in, and it is resulted with 4 bit output as r_rotate_out. The design of this sub module can be seen in (Figure8) but as can be seen that it is not useful due to the way the function works.

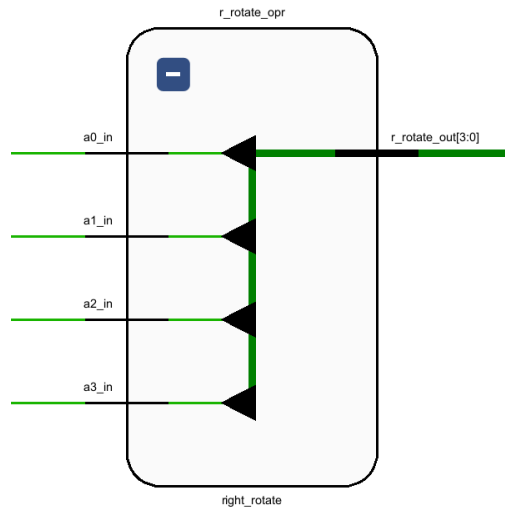


Figure8 : Right Rotate Opeation Schematics

The right logical shift operation submodule performs shifting each bit to the right but differently it inserts '0' to the left most bit. Right rotate operation takes 4 bit 'a' input as a0_in, a1_in, a2_in, a3_in, and it is resulted with 4 bit output as r_log_shift_out. The design of this sub module can be seen in (Figure9) but as can be seen that it is again not much useful due to the way the function works.

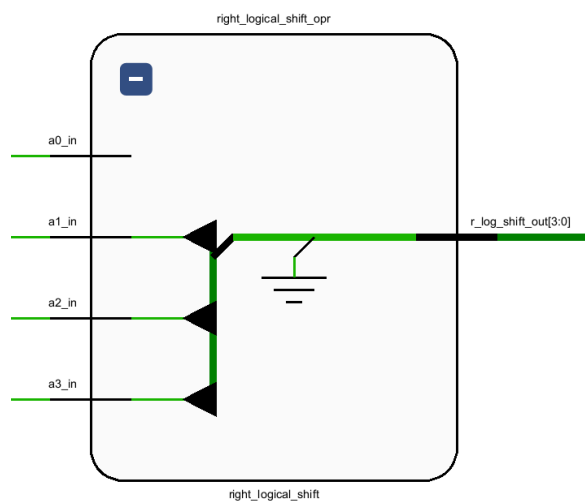


Figure9 : Right Logical Shift Operation Schematic

The submodule representing the bitwise complement is coded as complement operation. This operation takes 4 bit 'a' input as a0_in, a1_in, a2_in, a3_in and it is resulted with 4 bit output as comp_out. It uses 4 NOT gates to perform this operation for each bit and it give the result for each bit using NOT gates. Namely, this gives us to complements of input 'a' 's in binary numbers. The design of this sub module can be seen in (Figure10).

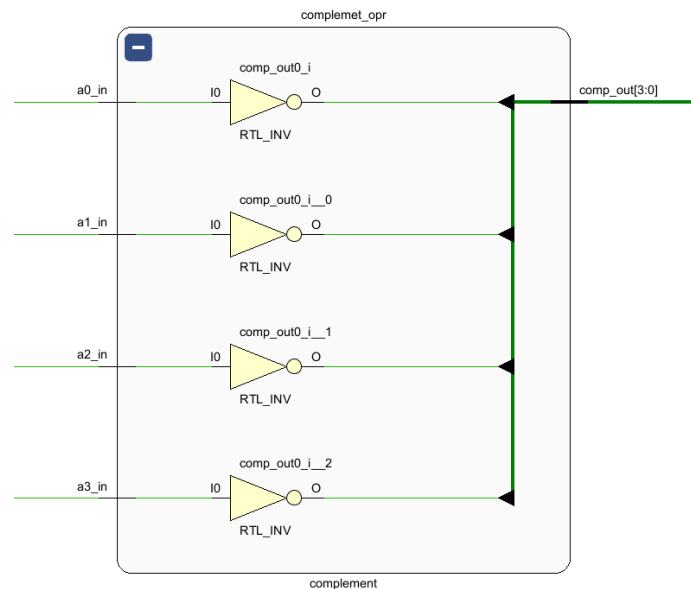


Figure10 : Complement Operation Schematic

Finally, we designed the ALU_calc_main module, which is our top module that makes all the sub modules we code functional and integrates them. This main module takes 4 bit a and b input as a0_in, a1_in, a2_in, a3_in, b0_in, b1_in, b2_in, b3_in and 3 bit selector input as sel and gives 5 bit output as out_result. It is designed to determine which function the ALU perform by selection from the user via the sel (selector) input and case process was used while doing this. Then two inputs, a())_ex_in and b())_ex_in, are taken and an output is resulted depending on the assigned function to the selector, and these outputs are displayed by the LED's on the Basys3 board. The operators assigned to the selector inputs and an example are given in (Table2).

Selector (sel)	Operator	External input a	External input b	Output result
000	Addition	1101	0010	1111
001	Substraction	1101	0010	1011
010	AND_Gate	1110	0111	0110
011	OR_Gate	1010	1001	1011
100	XOR_Gate	1101	0011	1110
101	Right Rotate	0011	--	1001
110	Right Logical Shift	0111	--	0011
111	Complement	1010	--	0101

Table2 : Selector Choices and Examples

The schematic design of the completed ALU can be observed in (Figure11). As can be clearly observed the 8 operator are exist and these outputs are going to an 8 to 1 multiplexer and it controlled by selector input.

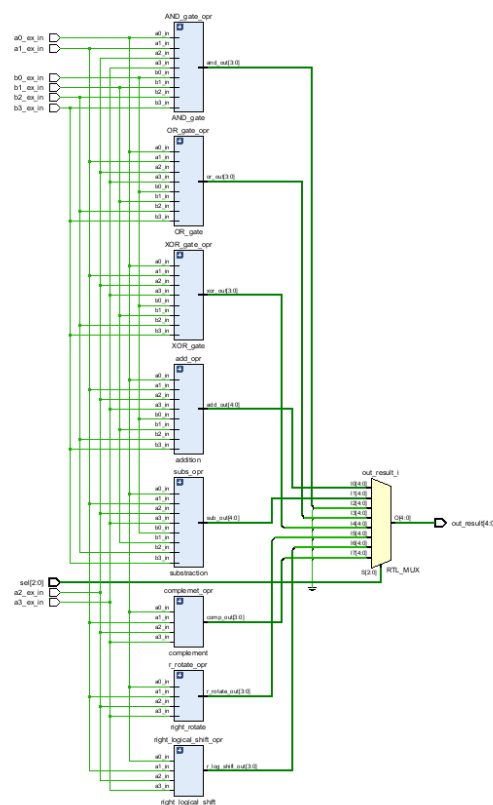


Figure11: ALU Schematic

METHODOLOGY:

First of all, we decided on the functions we would use to create the ALU and determined a design plan for these functions. We created submodules for each function on Vivado via VHDL code. We used the "component" function to call each submodule's operations to the top module of ALU. Then we added the selection code to them and completed the design. Then we wrote testbench code, our purpose here is to simulate and test whether the code we wrote works as expected, the created simulation can be observed in (Figure 12). To test the accuracy of our code, we also examined the RTL scheme (Figure11). Since the RTL scheme we created was created correctly and our simulation worked as expected, we wrote the constraint code, which includes which inputs and outputs represent which pins and LEDs on Basys3. Then we transferred our code to Basys3.

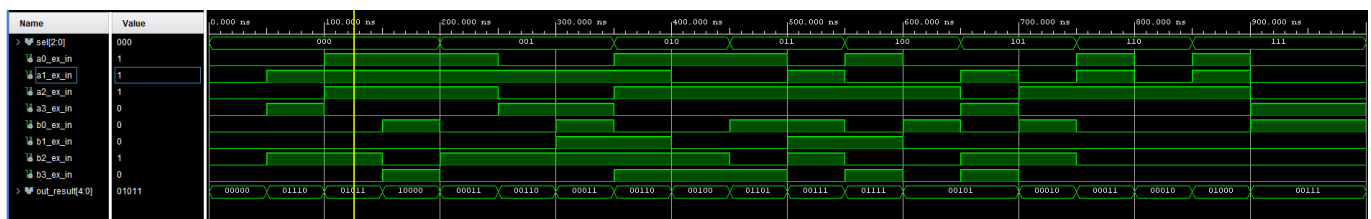


Figure12: Simulation

RESULTS:

The pin and LED configurations on Basys3 that we specified in the constraint file are as in (Table 3).

a0_ex_in	W2 Switch
a1_ex_in	U1 Switch
a2_ex_in	T1 Switch
a3_ex_in	R2 Switch
b0_ex_in	V2 Switch
b1_ex_in	T3 Switch
b2_ex_in	T2 Switch
b3_ex_in	R3 Switch
sel(0)	V17 Switch
sel(1)	V16 Switch
sel(2)	W16 Switch
out_result(0)	U3 LED
out_result(1)	P3 LED
out_result(2)	N3 LED
out_result(3)	P1 LED
out_result(4)	L1 LED

Table3 : Pin and LED Configurations

Then we tested some cases on Basys3

Case 1

a_ex_in	1101
b_ex_in	0010
sel	000
out_result	1111
Expected result	01111

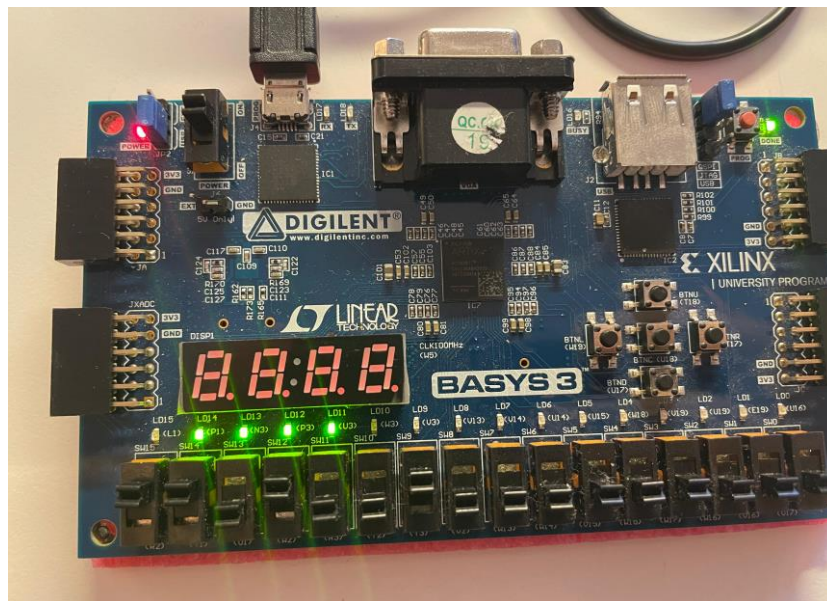


Figure 13.1 : Case 1

Case 2

a_ex_in	1101
b_ex_in	0010
sel	001
out_result	1011
Expected result	01011

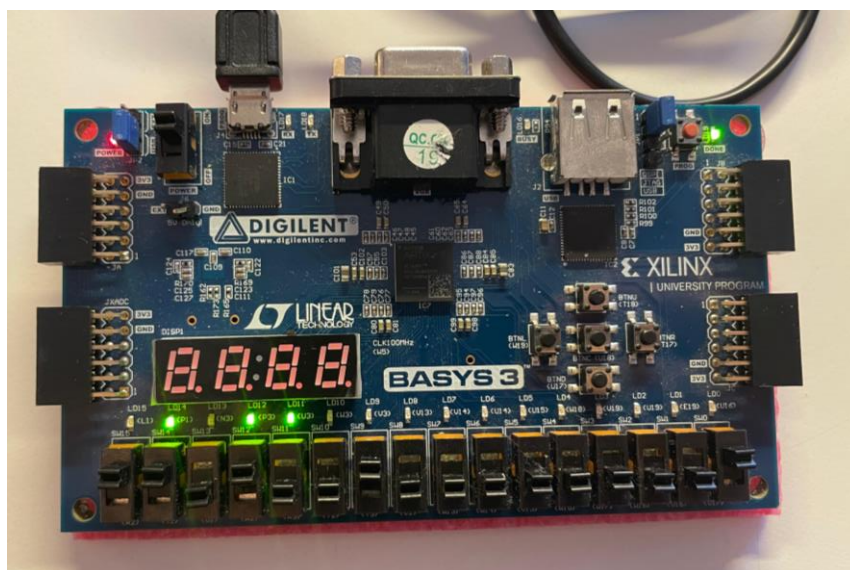


Figure 13.2 : Case 2

Case 3

a_ex_in	1110
b_ex_in	0111
sel	010
out_result	0110
Expected result	00110

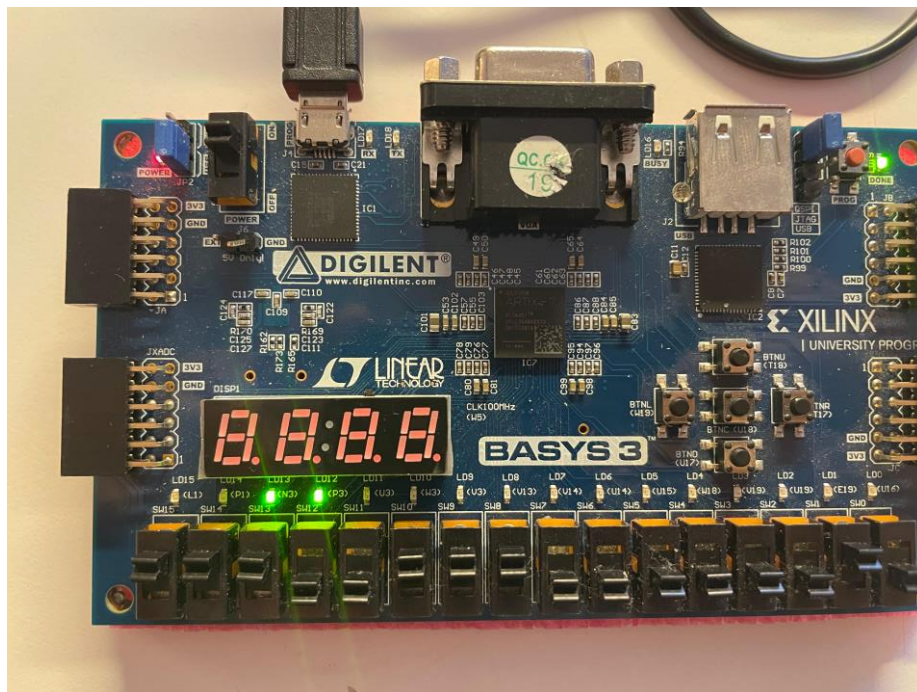


Figure 13.3 : Case 3

Case 4

a_ex_in	1010
b_ex_in	1001
sel	011
out_result	1011
Expected result	01011

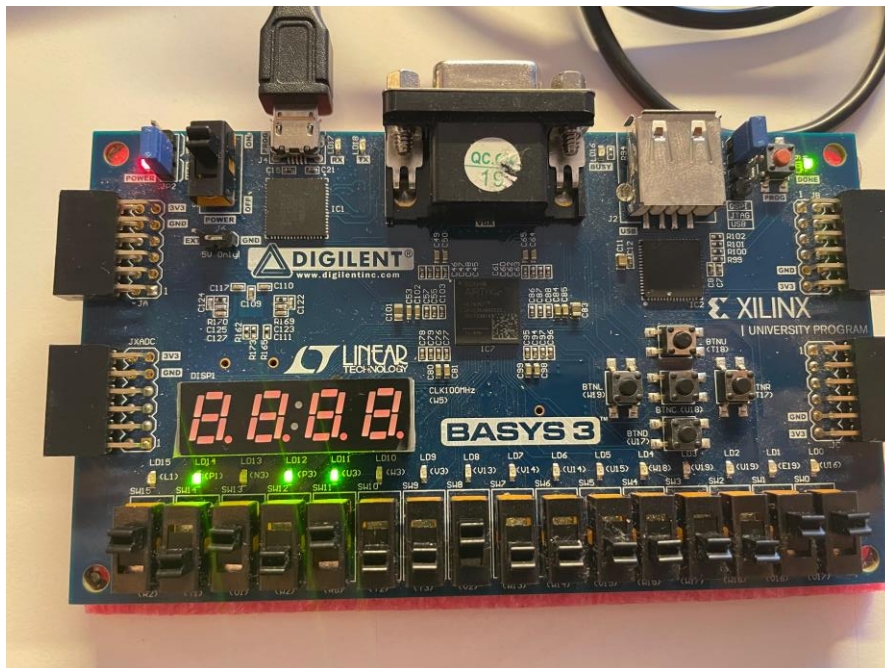


Figure 13.4 : Case 4

Case 5

a_ex_in	1101
b_ex_in	0011
sel	100
out_result	1110
Expected result	01110

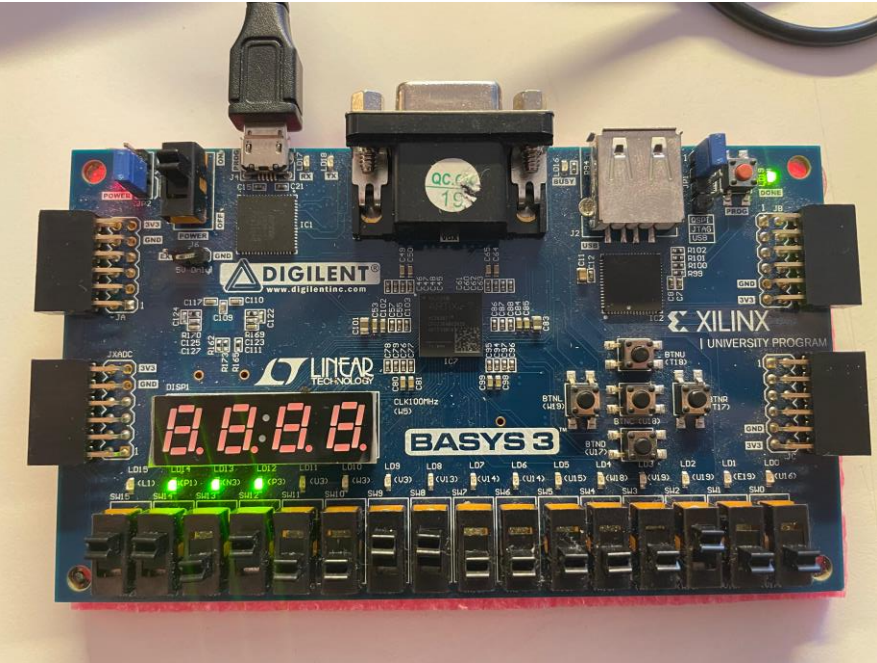


Figure 13.5 : Case 5

Case 6

a_ex_in	0011
sel	101
out_result	1001
Expected result	01001

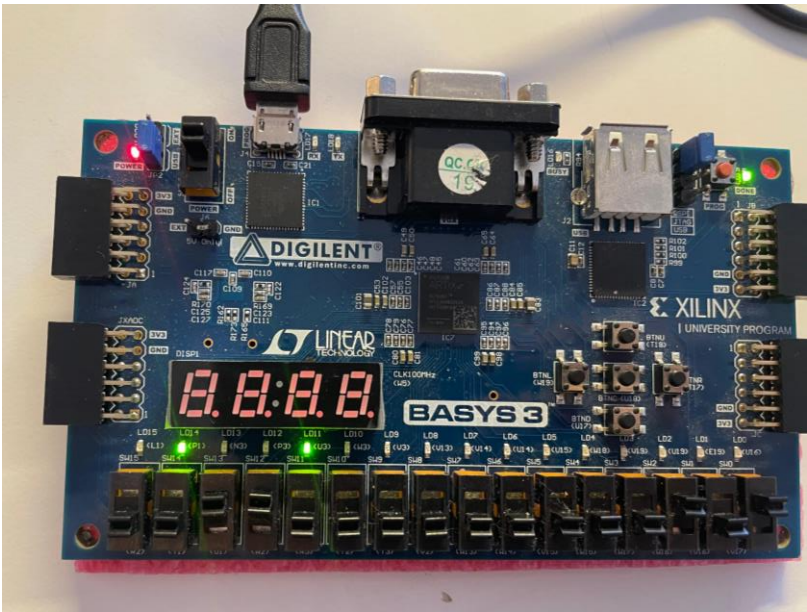


Figure 13.6 : Case 6

Case 7

a_ex_in	0111
sel	110
out_result	0011
Expected result	00011

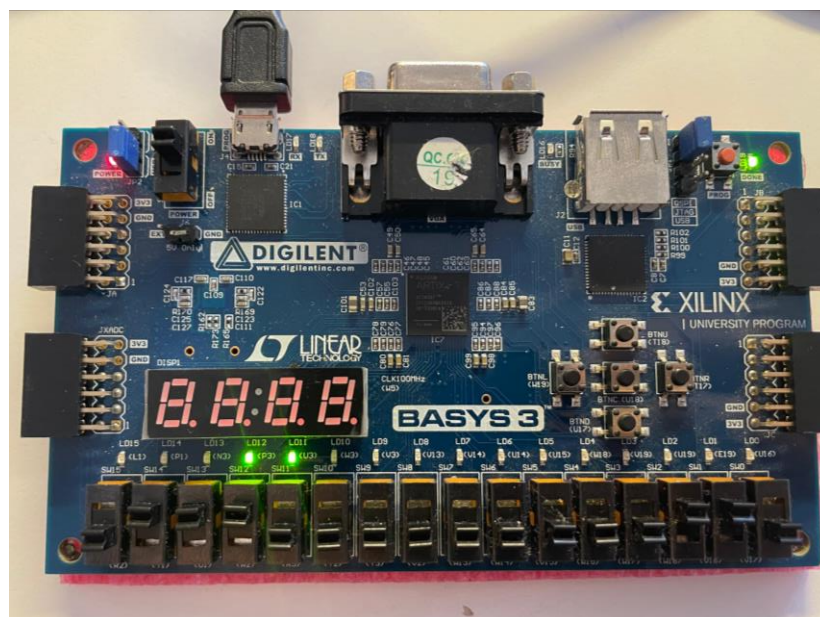


Figure 13.7 : Case 7

Case 8

a_ex_in	1010
sel	111
out_result	0101
Expected result	00101

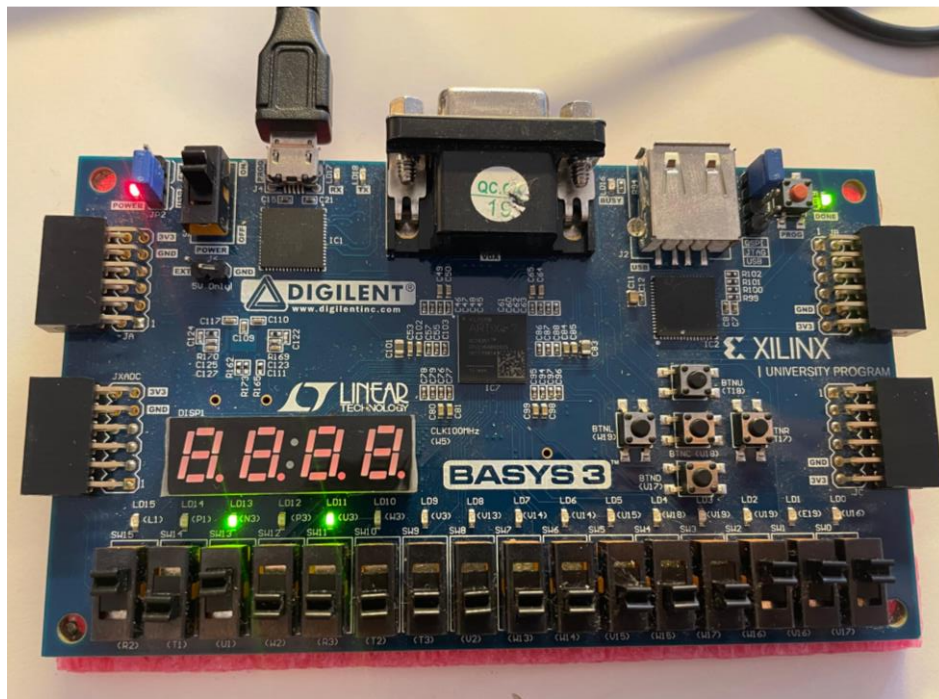


Figure 13.8 : Case 8

The cases above show that the ALU is working as expected and the outputs match the simulation.

CONCLUSION:

In this laboratory project, we coded an arithmetic logic unit (ALU) that processes 4 bit inputs and can perform 8 functions on Basys3 with VHDL. We have successfully created all submodules and connected them to the top main module. Our code worked successfully at the end of the process, but many syntax errors were encountered during the process, but these errors were overcome and resolved with research. Thanks to this project, we did more research on the logic of VHDL coding and learned information that will form the basis of many complex structures.

APPENDIX:

ALU_calc_main.vhd

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;


entity ALU_calc_main is
Port(
a0_ex_in : in std_logic;
a1_ex_in : in std_logic;
a2_ex_in : in std_logic;
a3_ex_in : in std_logic;
b0_ex_in : in std_logic;
b1_ex_in : in std_logic;
b2_ex_in : in std_logic;
b3_ex_in : in std_logic;
sel: in std_logic_vector(2 downto 0);
out_result: out std_logic_vector(4 downto 0));

end ALU_calc_main;


architecture Behavioral of ALU_calc_main is

component addition is
Port (
a0_in : in std_logic;
a1_in : in std_logic;
a2_in : in std_logic;
a3_in : in std_logic;
```

```
b0_in : in std_logic;  
b1_in : in std_logic;  
b2_in : in std_logic;  
b3_in : in std_logic;  
add_out : out std_logic_vector(4 downto 0)  
);  
end component;
```

component subtraction is

```
Port (  
a0_in : in std_logic;  
a1_in : in std_logic;  
a2_in : in std_logic;  
a3_in : in std_logic;  
b0_in : in std_logic;  
b1_in : in std_logic;  
b2_in : in std_logic;  
b3_in : in std_logic;  
sub_out : out std_logic_vector(4 downto 0)  
);  
end component;
```

component AND_gate is

```
Port (  
a0_in : in std_logic;  
a1_in : in std_logic;  
a2_in : in std_logic;  
a3_in : in std_logic;  
b0_in : in std_logic;  
b1_in : in std_logic;  
b2_in : in std_logic;  
b3_in : in std_logic;  
and_out : out std_logic_vector(3 downto 0)
```

```
);
```

```
end component;
```

```
component OR_gate is
```

```
Port (
```

```
a0_in : in std_logic;
```

```
a1_in : in std_logic;
```

```
a2_in : in std_logic;
```

```
a3_in : in std_logic;
```

```
b0_in : in std_logic;
```

```
b1_in : in std_logic;
```

```
b2_in : in std_logic;
```

```
b3_in : in std_logic;
```

```
or_out : out std_logic_vector(3 downto 0)
```

```
);
```

```
end component;
```

```
component XOR_gate is
```

```
Port (
```

```
a0_in : in std_logic;
```

```
a1_in : in std_logic;
```

```
a2_in : in std_logic;
```

```
a3_in : in std_logic;
```

```
b0_in : in std_logic;
```

```
b1_in : in std_logic;
```

```
b2_in : in std_logic;
```

```
b3_in : in std_logic;
```

```
xor_out : out std_logic_vector(3 downto 0)
```

```
);
```

```
end component;
```

```
component right_rotate is
```

```
Port (
```

```

a0_in : in std_logic;
a1_in : in std_logic;
a2_in : in std_logic;
a3_in : in std_logic;
r_rotate_out : out std_logic_vector(3 downto 0)
);
end component;

```

component right_logical_shift is

```

Port (
a0_in : in std_logic;
a1_in : in std_logic;
a2_in : in std_logic;
a3_in : in std_logic;
r_log_shift_out : out std_logic_vector(3 downto 0)
);
end component;

```

component complement is

```

Port (
a0_in : in std_logic;
a1_in : in std_logic;
a2_in : in std_logic;
a3_in : in std_logic;
comp_out : out std_logic_vector(3 downto 0)
);
end component;

```

```

signal a0_in,a1_in,a2_in,a3_in,a4_in,a5_in,a6_in,a7_in : std_logic;
signal opr0,opr1 : std_logic_vector(4 downto 0);
signal opr2,opr3,opr4,opr5,opr6,opr7 : std_logic_vector(3 downto 0);

```

```
begin
```

```
add_opr : addition
```

```
port map(
```

```
a0_in => a0_ex_in,
```

```
a1_in => a1_ex_in,
```

```
a2_in => a2_ex_in,
```

```
a3_in => a3_ex_in,
```

```
b0_in => b0_ex_in,
```

```
b1_in => b1_ex_in,
```

```
b2_in => b2_ex_in,
```

```
b3_in => b3_ex_in,
```

```
add_out => opr0);
```

```
subs_opr : subtraction
```

```
port map(
```

```
a0_in => a0_ex_in,
```

```
a1_in => a1_ex_in,
```

```
a2_in => a2_ex_in,
```

```
a3_in => a3_ex_in,
```

```
b0_in => b0_ex_in,
```

```
b1_in => b1_ex_in,
```

```
b2_in => b2_ex_in,
```

```
b3_in => b3_ex_in,
```

```
sub_out => opr1);
```

```
AND_gate_opr : AND_gate
```

```
port map(
```

```
a0_in => a0_ex_in,
```

```
a1_in => a1_ex_in,
```

```
a2_in => a2_ex_in,
```

```
a3_in => a3_ex_in,
```

```
b0_in => b0_ex_in,  
b1_in => b1_ex_in,  
b2_in => b2_ex_in,  
b3_in => b3_ex_in,  
and_out => opr2);
```

```
OR_gate_opr : OR_gate
```

```
port map(  
a0_in => a0_ex_in,  
a1_in => a1_ex_in,  
a2_in => a2_ex_in,  
a3_in => a3_ex_in,  
b0_in => b0_ex_in,  
b1_in => b1_ex_in,  
b2_in => b2_ex_in,  
b3_in => b3_ex_in,  
or_out => opr3);
```

```
XOR_gate_opr : XOR_gate
```

```
port map(  
a0_in => a0_ex_in,  
a1_in => a1_ex_in,  
a2_in => a2_ex_in,  
a3_in => a3_ex_in,  
b0_in => b0_ex_in,  
b1_in => b1_ex_in,  
b2_in => b2_ex_in,  
b3_in => b3_ex_in,  
xor_out => opr4);
```

```
r_rotate_opr : right_rotate
```

```
port map(  
a0_in => a0_ex_in,
```

```
a1_in => a1_ex_in,  
a2_in => a2_ex_in,  
a3_in => a3_ex_in,  
r_rotate_out => opr5);
```

```
right_logical_shift_opr : right_logical_shift  
port map(  
a0_in => a0_ex_in,  
a1_in => a1_ex_in,  
a2_in => a2_ex_in,  
a3_in => a3_ex_in,  
r_log_shift_out => opr6);
```

```
complemet_opr : complement  
port map(  
a0_in => a0_ex_in,  
a1_in => a1_ex_in,  
a2_in => a2_ex_in,  
a3_in => a3_ex_in,  
comp_out => opr7);
```

```
process(sel,opr0,opr1,opr2,opr3,opr4,opr5,opr6,opr7) is  
begin
```

```
out_result <= "00000" ;
```

```
case sel is  
when "000" => out_result <= opr0;  
when "001" => out_result <= opr1;  
when "010" => out_result(3 downto 0) <= opr2;  
when "011" => out_result(3 downto 0) <= opr3;  
when "100" => out_result(3 downto 0) <= opr4;  
when "101" => out_result(3 downto 0) <= opr5;
```



```
when "110" => out_result(3 downto 0) <= opr6;
when "111" => out_result(3 downto 0) <= opr7;
when others => out_result <= "00000";
end case;
end process;

end Behavioral;
```

half_adder.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity half_adder is
    Port(
        in_halad_a : in std_logic ;
        in_halad_b : in std_logic ;
        out_halad_sum : out std_logic ;
        out_halad_car : out std_logic );
end half_adder;

architecture Behavioral of half_adder is

begin

    out_halad_sum <= in_halad_a xor in_halad_b;
    out_halad_car <= in_halad_a and in_halad_b;

end Behavioral;
```

full_adder.vhd

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;


entity full_adder is
    Port(
        in_fulad_c : in std_logic;
        in_fulad_a : in std_logic ;
        in_fulad_b : in std_logic ;
        out_fulad_sum : out std_logic ;
        out_fulad_car : out std_logic );

end full_adder;


architecture Behavioral of full_adder is

begin

    out_fulad_sum <= in_fulad_c xor in_fulad_a xor in_fulad_b;
    out_fulad_car <= (in_fulad_c and in_fulad_a) or (in_fulad_c and in_fulad_b) or (in_fulad_a and in_fulad_b);

end Behavioral;
```

add_opr.vhd

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;


entity addition is
    Port(
        a0_in : in std_logic;
        a1_in : in std_logic;
        a2_in : in std_logic;
        a3_in : in std_logic;
```

```
b0_in : in std_logic;
b1_in : in std_logic;
b2_in : in std_logic;
b3_in : in std_logic;
add_out : out std_logic_vector(4 downto 0));

end addition;
```

architecture Behavioral of addition is

component half_adder is

```
Port(
in_halad_a : in std_logic ;
in_halad_b : in std_logic ;
out_halad_sum : out std_logic ;
out_halad_car : out std_logic);
end component;
```

component full_adder is

```
Port(
in_fulad_a : in std_logic;
in_fulad_b : in std_logic ;
in_fulad_c : in std_logic ;
out_fulad_sum : out std_logic ;
out_fulad_car : out STD_logic);
end component;
```

```
signal c: std_logic_vector (2 downto 0);
```

```
begin
```

```
half_adder_0: half_adder
```

```
Port map(
```

```
in_halad_a => a0_in,  
in_halad_b => b0_in,  
out_halad_sum => add_out(0),  
out_halad_car => c(0));
```

```
full_adder_0 : full_adder  
Port map(  
in_fulad_a => a1_in,  
in_fulad_b => b1_in,  
in_fulad_c => c(0),  
out_fulad_sum => add_out(1),  
out_fulad_car => c(1));
```

```
full_adder_1 : full_adder  
Port map(  
in_fulad_a => a2_in,  
in_fulad_b => b2_in,  
in_fulad_c => c(1),  
out_fulad_sum => add_out(2),  
out_fulad_car => c(2));
```

```
full_adder_2 : full_adder  
Port map(  
in_fulad_a => a3_in,  
in_fulad_b => b3_in,  
in_fulad_c => c(2),  
out_fulad_sum => add_out(3),  
out_fulad_car => add_out(4));
```

```
end Behavioral;
```

subs_opr.vhd

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity subtraction is

Port(

a0_in : in std_logic;

a1_in : in std_logic;

a2_in : in std_logic;

a3_in : in std_logic;

b0_in : in std_logic;

b1_in : in std_logic;

b2_in : in std_logic;

b3_in : in std_logic;

sub_out : out std_logic_vector(4 downto 0));

end subtraction;

architecture Behavioral of subtraction is

component full_adder is

Port(

in_fulad_a : in std_logic;

in_fulad_b : in std_logic ;

in_fulad_c : in std_logic ;

out_fulad_sum : out std_logic ;

out_fulad_car : out STD_logic);

end component;

signal c: std_logic_vector (3 downto 0);

signal inv_b : std_logic_vector(3 downto 0);

begin

```
inv_b(0) <= not b0_in ;  
inv_b(1) <= not b1_in ;  
inv_b(2) <= not b2_in ;  
inv_b(3) <= not b3_in ;
```

```
full_adder_0 : full_adder  
Port map(  
  in_fulad_a => a0_in,  
  in_fulad_b => inv_b(0),  
  in_fulad_c => '1',  
  out_fulad_sum => sub_out(0),  
  out_fulad_car => c(0));
```

```
full_adder_1 : full_adder  
Port map(  
  in_fulad_a => a1_in,  
  in_fulad_b => inv_b(1),  
  in_fulad_c => c(0),  
  out_fulad_sum => sub_out(1),  
  out_fulad_car => c(1));
```

```
full_adder_2 : full_adder  
Port map(  
  in_fulad_a => a2_in,  
  in_fulad_b => inv_b(2),  
  in_fulad_c => c(1),  
  out_fulad_sum => sub_out(2),  
  out_fulad_car => c(2));
```

```
full_adder_3 : full_adder  
Port map(  
  in_fulad_a => a3_in,  
  in_fulad_b => inv_b(3),  
  in_fulad_c => c(2),  
  out_fulad_sum => sub_out(3),  
  out_fulad_car => c(3));
```

```

in_fulad_a => a3_in,
in_fulad_b => inv_b(3),
in_fulad_c => c(2),
out_fulad_sum => sub_out(3),
out_fulad_car => c(3));
sub_out(4) <= not c(3);

end Behavioral;

```

AND_gate_opr.vhd

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity AND_gate is
Port (
a0_in : in std_logic;
a1_in : in std_logic;
a2_in : in std_logic;
a3_in : in std_logic;
b0_in : in std_logic;
b1_in : in std_logic;
b2_in : in std_logic;
b3_in : in std_logic;
and_out : out std_logic_vector(3 downto 0));

end AND_gate;

architecture Behavioral of AND_gate is

begin

and_out(0) <= a0_in and b0_in;
and_out(1) <= a1_in and b1_in;

```

```
and_out(2) <= a2_in and b2_in;  
and_out(3) <= a3_in and b3_in;
```

```
end Behavioral;
```

OR_gate_opr.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity OR_gate is
```

```
Port (
```

```
a0_in : in std_logic;
```

```
a1_in : in std_logic;
```

```
a2_in : in std_logic;
```

```
a3_in : in std_logic;
```

```
b0_in : in std_logic;
```

```
b1_in : in std_logic;
```

```
b2_in : in std_logic;
```

```
b3_in : in std_logic;
```

```
or_out : out std_logic_vector(3 downto 0));
```

```
end OR_gate;
```

```
architecture Behavioral of OR_gate is
```

```
begin
```

```
or_out(0) <= a0_in or b0_in;
```

```
or_out(1) <= a1_in or b1_in;
```

```
or_out(2) <= a2_in or b2_in;
```

```
or_out(3) <= a3_in or b3_in;
```

```
end Behavioral;
```


XOR_gate_opr.vhd

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;


entity XOR_gate is
    Port (
        a0_in : in std_logic;
        a1_in : in std_logic;
        a2_in : in std_logic;
        a3_in : in std_logic;
        b0_in : in std_logic;
        b1_in : in std_logic;
        b2_in : in std_logic;
        b3_in : in std_logic;
        xor_out : out std_logic_vector(3 downto 0));

end XOR_gate;


architecture Behavioral of XOR_gate is

begin

    xor_out(0) <= a0_in xor b0_in;
    xor_out(1) <= a1_in xor b1_in;
    xor_out(2) <= a2_in xor b2_in;
    xor_out(3) <= a3_in xor b3_in;

end Behavioral;
```

r_rotate_opr.vhd

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;
```

```
entity right_rotate is
Port(
a0_in : in std_logic;
a1_in : in std_logic;
a2_in : in std_logic;
a3_in : in std_logic;
r_rotate_out : out std_logic_vector(3 downto 0));

end right_rotate;
```

architecture Behavioral of right_rotate is

```
begin

r_rotate_out(0) <= a1_in;
r_rotate_out(1) <= a2_in;
r_rotate_out(2) <= a3_in;
r_rotate_out(3) <= a0_in;
```

```
end Behavioral;
```

right_logical_shift_opr.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity right_logical_shift is
```

```
Port(
a0_in : in std_logic;
a1_in : in std_logic;
a2_in : in std_logic;
a3_in : in std_logic;
```

```
r_log_shift_out : out std_logic_vector(3 downto 0));
```

```
end right_logical_shift;
```

architecture Behavioral of right_logical_shift is

```
begin
```

```
    r_log_shift_out(0) <= a1_in;
```

```
    r_log_shift_out(1) <= a2_in;
```

```
    r_log_shift_out(2) <= a3_in;
```

```
    r_log_shift_out(3) <= '0';
```

```
end Behavioral;
```

complement_opr.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

entity complement is

```
Port (
```

```
    a0_in : in std_logic;
```

```
    a1_in : in std_logic;
```

```
    a2_in : in std_logic;
```

```
    a3_in : in std_logic;
```

```
    comp_out : out std_logic_vector(3 downto 0));
```

```
end complement;
```

architecture Behavioral of complement is

```
begin
```

```
    comp_out(0) <= not a0_in ;
```

```
comp_out(1) <= not a1_in ;  
comp_out(2) <= not a2_in ;  
comp_out(3) <= not a3_in ;
```

```
end Behavioral;
```

ALU_testbench.vhd

```
library IEEE;
```

```
use IEEE.STD_LOGIC_1164.ALL;
```

```
entity ALU_testbench is
```

```
end ALU_testbench;
```

```
architecture Behavioral of ALU_testbench is
```

```
component ALU_calc_main is
```

```
Port(  
  a0_ex_in : in std_logic;  
  a1_ex_in : in std_logic;  
  a2_ex_in : in std_logic;  
  a3_ex_in : in std_logic;  
  b0_ex_in : in std_logic;  
  b1_ex_in : in std_logic;  
  b2_ex_in : in std_logic;  
  b3_ex_in : in std_logic;  
  sel: in std_logic_vector(2 downto 0);  
  out_result: out std_logic_vector(4 downto 0));  
end component;
```

```
signal sel : std_logic_vector (2 downto 0):= "000";
```

```
signal a0_ex_in,a1_ex_in,a2_ex_in,a3_ex_in,b0_ex_in,b1_ex_in,b2_ex_in,b3_ex_in : std_logic := '0';
```

```
signal out_result : std_logic_vector (4 downto 0) := "00000";
```

```
begin
```

```
T1: ALU_calc_main
```

```
Port map(
```

```
sel => sel,
```

```
a0_ex_in => a0_ex_in,
```

```
a1_ex_in => a1_ex_in,
```

```
a2_ex_in => a2_ex_in,
```

```
a3_ex_in => a3_ex_in,
```

```
b0_ex_in => b0_ex_in,
```

```
b1_ex_in => b1_ex_in,
```

```
b2_ex_in => b2_ex_in,
```

```
b3_ex_in => b3_ex_in,
```

```
out_result => out_result);
```

```
testbench_alu_1 : process
```

```
begin
```

```
wait for 50ns;
```

```
sel <= "000";
```

```
a0_ex_in <= '0';
```

```
a1_ex_in <= '1';
```

```
a2_ex_in <= '0';
```

```
a3_ex_in <= '1';
```

```
b0_ex_in <= '0';
```

```
b1_ex_in <= '0';
```

```
b2_ex_in <= '1';
```

```
b3_ex_in <= '0';
```

```
wait for 50ns;
```

```
sel <= "000";
```

```
a0_ex_in <= '1';  
a1_ex_in <= '1';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '0';  
b1_ex_in <= '0';  
b2_ex_in <= '1';  
b3_ex_in <= '0';  
wait for 50ns;
```

```
sel <= "000";  
a0_ex_in <= '1';  
a1_ex_in <= '1';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '1';  
b1_ex_in <= '0';  
b2_ex_in <= '0';  
b3_ex_in <= '1';  
wait for 50ns;
```

```
sel <= "001";  
a0_ex_in <= '1';  
a1_ex_in <= '1';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '0';  
b1_ex_in <= '0';  
b2_ex_in <= '1';  
b3_ex_in <= '0';  
wait for 50ns;
```

```
sel <= "001";  
a0_ex_in <= '0';  
a1_ex_in <= '1';  
a2_ex_in <= '0';  
a3_ex_in <= '1';  
b0_ex_in <= '0';  
b1_ex_in <= '0';  
b2_ex_in <= '1';  
b3_ex_in <= '0';  
wait for 50ns;
```

```
sel <= "001";  
a0_ex_in <= '0';  
a1_ex_in <= '1';  
a2_ex_in <= '0';  
a3_ex_in <= '1';  
b0_ex_in <= '1';  
b1_ex_in <= '1';  
b2_ex_in <= '1';  
b3_ex_in <= '0';  
wait for 50ns;
```

```
sel <= "010";  
a0_ex_in <= '1';  
a1_ex_in <= '1';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '0';  
b1_ex_in <= '1';  
b2_ex_in <= '1';
```

```
b3_ex_in <= '1';  
wait for 50ns;
```

```
sel <= "010";  
a0_ex_in <= '1';  
a1_ex_in <= '0';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '0';  
b1_ex_in <= '0';  
b2_ex_in <= '1';  
b3_ex_in <= '1';  
wait for 50ns;
```

```
sel <= "011";  
a0_ex_in <= '1';  
a1_ex_in <= '0';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '1';  
b1_ex_in <= '0';  
b2_ex_in <= '0';  
b3_ex_in <= '1';  
wait for 50ns;
```

```
sel <= "011";  
a0_ex_in <= '0';  
a1_ex_in <= '1';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '1';
```



```
b1_ex_in <= '1';  
b2_ex_in <= '1';  
b3_ex_in <= '0';  
wait for 50ns;
```

```
sel <= "100";  
a0_ex_in <= '1';  
a1_ex_in <= '0';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '0';  
b1_ex_in <= '1';  
b2_ex_in <= '0';  
b3_ex_in <= '1';  
wait for 50ns;
```

```
sel <= "100";  
a0_ex_in <= '0';  
a1_ex_in <= '0';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '1';  
b1_ex_in <= '0';  
b2_ex_in <= '0';  
b3_ex_in <= '0';  
wait for 50ns;
```

```
sel <= "101";  
a0_ex_in <= '0';  
a1_ex_in <= '1';  
a2_ex_in <= '0';
```

```
a3_ex_in <= '1';  
b0_ex_in <= '0';  
b1_ex_in <= '0';  
b2_ex_in <= '1';  
b3_ex_in <= '1';  
wait for 50ns;
```

```
sel <= "101";  
a0_ex_in <= '0';  
a1_ex_in <= '0';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '1';  
b1_ex_in <= '0';  
b2_ex_in <= '1';  
b3_ex_in <= '0';  
wait for 50ns;
```

```
sel <= "110";  
a0_ex_in <= '1';  
a1_ex_in <= '1';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '0';  
b1_ex_in <= '0';  
b2_ex_in <= '0';  
b3_ex_in <= '0';  
wait for 50ns;
```

```
sel <= "110";  
a0_ex_in <= '0';
```

```
a1_ex_in <= '0';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '0';  
b1_ex_in <= '0';  
b2_ex_in <= '0';  
b3_ex_in <= '0';  
wait for 50ns;
```

```
sel <= "111";  
a0_ex_in <= '1';  
a1_ex_in <= '1';  
a2_ex_in <= '1';  
a3_ex_in <= '0';  
b0_ex_in <= '0';  
b1_ex_in <= '0';  
b2_ex_in <= '0';  
b3_ex_in <= '0';  
wait for 50ns;
```

```
sel <= "111";  
a0_ex_in <= '0';  
a1_ex_in <= '0';  
a2_ex_in <= '0';  
a3_ex_in <= '1';  
b0_ex_in <= '1';  
b1_ex_in <= '0';  
b2_ex_in <= '0';  
b3_ex_in <= '0';  
wait for 50ns;
```

end process;

end Behavioral;

ALU_cons.xdc

```
set_property PACKAGE_PIN W2 [get_ports {a0_ex_in}]
set_property IOSTANDARD LVCMOS33 [get_ports {a0_ex_in}]
set_property PACKAGE_PIN U1 [get_ports {a1_ex_in}]
set_property IOSTANDARD LVCMOS33 [get_ports {a1_ex_in}]
set_property PACKAGE_PIN T1 [get_ports {a2_ex_in}]
set_property IOSTANDARD LVCMOS33 [get_ports {a2_ex_in}]
set_property PACKAGE_PIN R2 [get_ports {a3_ex_in}]
set_property IOSTANDARD LVCMOS33 [get_ports {a3_ex_in}]
```

```
set_property PACKAGE_PIN V2 [get_ports {b0_ex_in}]
set_property IOSTANDARD LVCMOS33 [get_ports {b0_ex_in}]
set_property PACKAGE_PIN T3 [get_ports {b1_ex_in}]
set_property IOSTANDARD LVCMOS33 [get_ports {b1_ex_in}]
set_property PACKAGE_PIN T2 [get_ports {b2_ex_in}]
set_property IOSTANDARD LVCMOS33 [get_ports {b2_ex_in}]
set_property PACKAGE_PIN R3 [get_ports {b3_ex_in}]
set_property IOSTANDARD LVCMOS33 [get_ports {b3_ex_in}]
```

```
set_property PACKAGE_PIN V17 [get_ports {sel[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sel[0]}]
set_property PACKAGE_PIN V16 [get_ports {sel[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sel[1]}]
set_property PACKAGE_PIN W16 [get_ports {sel[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sel[2]}]
```

```
set_property PACKAGE_PIN U3 [get_ports {out_result[0]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {out_result[0]}]
set_property PACKAGE_PIN P3 [get_ports {out_result[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {out_result[1]}]
set_property PACKAGE_PIN N3 [get_ports {out_result[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {out_result[2]}]
set_property PACKAGE_PIN P1 [get_ports {out_result[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {out_result[3]}]
set_property PACKAGE_PIN L1 [get_ports {out_result[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {out_result[4]}]
```